

LAPORAN TUGAS BESAR 2 IF2211 STRATEGI ALGORITMA
*Pemanfaatan Algoritma BFS dan DFS dalam Mekanisme
Penelusuran CSS pada pohon Document Object Model*
Semester II Tahun Ajaran 2025/2026



Disusun oleh:

Jonathan Alveraldo Bangun	(13524120)
Zahran Alvan Putra Winarko	(13524124)
Dika Pramudya Nugraha	(13524132)

Laboratorium Ilmu dan Rekayasa Komputasi
Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung
2026

Kata Pengantar

Dokumen Laporan Tugas Besar 2 ini disusun mahasiswa Teknik Informatika angkatan 2024. Adapun susunan anggota kelompok kami secara detail sebagai berikut:

1. Jonathan Alveraldo Bangun (NIM: 13524120)
2. Zahran Alvan Putra Winarko (NIM: 13524124)
3. Dika Pramudya Nugraha (NIM: 13524132)

Daftar Isi

Kata Pengantar.....	1
Daftar Isi.....	2
BAB 1.....	3
DESKRIPSI TUGAS.....	3
1.1 Document Object Model (DOM) dan Struktur HTML.....	3
1.2 Tujuan Tugas.....	3
1.3 Spesifikasi Wajib.....	3
BAB 2.....	5
LANDASAN TEORI.....	5
2.1 Document Object Model (DOM) dan Struktur HTML.....	5
2.2 Mekanisme Leksikal dan Sintaksis HTML Parsing.....	6
2.3 Pola Pencocokan CSS Selector dan Operasi Combinator.....	7
2.4 Strategi Penelusuran Breadth-First Search (BFS).....	8
2.5 Strategi Penelusuran Depth-First Search (DFS).....	9
BAB 3.....	10
APLIKASI BFS DAN DFS PADA PENELUSURAN POHON DOM.....	10
3.1 Representasi DOM sebagai Struktur Pohon (Tree).....	10
3.2 Penelusuran Menggunakan Breadth-First Search (BFS).....	10
3.3 Paralelisasi Breadth-First Search (Parallel BFS).....	11
3.4 Paralelisasi Menggunakan Depth-First Search (DFS).....	11
3.5 Aplikasi Selektor dalam Penelusuran.....	12
BAB 4.....	13
IMPLEMENTASI DAN PENGUJIAN.....	13
4.1 Implementasi.....	13
4.2 Pengujian.....	17
BAB 5.....	40
KESIMPULAN DAN SARAN.....	40
5.1 Kesimpulan.....	40
5.2 Saran.....	40
Untuk pengembangan aplikasi di masa mendatang, berikut adalah beberapa saran dari kami yang dapat dipertimbangkan:.....	40
LAMPIRAN.....	41
PEMBAGIAN TUGAS.....	42
PERNYATAAN TIDAK MELAKUKAN KECURANGAN.....	43
DAFTAR PUSTAKA.....	44

BAB 1

DESKRIPSI TUGAS

Tugas Besar 2 mata kuliah IF2211 Strategi Algoritma Semester II tahun 2025/2026 ini berfokus pada topik “*Pemanfaatan Algoritma BFS dan DFS dalam Mekanisme Penelusuran CSS pada Pohon Document Object Model*”.

1.1 Document Object Model (DOM) dan Struktur HTML

Document Object Model (DOM) adalah representasi terstruktur dari dokumen HTML dalam bentuk pohon (*tree*) yang memungkinkan manipulasi elemen pada halaman web. Untuk mengontrol visualisasi elemen-elemen ini, digunakan *Cascading Style Sheets* (CSS) yang menempelkan aturan visualisasi melalui *CSS Selector*. Tugas besar ini bertujuan untuk menguji penerapan algoritma penelusuran graf, yaitu Breadth First Search (BFS) dan Depth First Search (DFS), dalam menjelajahi struktur pohon DOM untuk menemukan elemen yang sesuai dengan *CSS Selector* yang diberikan.

1.2 Tujuan Tugas

Tujuan utama dari Tugas Besar ini adalah membuat sebuah aplikasi yang dapat mengimplementasikan algoritma Breadth First Search (BFS) dan Depth First Search (DFS) untuk melakukan penelusuran elemen, serta mengaplikasikan algoritma tersebut untuk mencari elemen pada struktur DOM berdasarkan input *CSS selector* tertentu.

1.3 Spesifikasi Wajib

Membuat aplikasi berbasis web yang berfungsi sebagai *traversal* pohon HTML dan harus memenuhi spesifikasi berikut:

1.3.1 Teknologi dan Implementasi

- Aplikasi harus berbasis web, dengan format frontend bebas dan backend yang diimplementasikan dalam bahasa Go, C#, Rust, atau Typescript.
- Aplikasi wajib mengimplementasikan algoritma BFS dan DFS untuk penelusuran CSS pada pohon DOM.

1.3.2 Fungsionalitas Input

- Aplikasi harus mampu menerima input URL *website* atau teks HTML yang dimasukkan secara langsung dari pengguna.
- Aplikasi harus mampu menerima input berikut dari pengguna Pilihan algoritma traversal yang akan digunakan (BFS / DFS) dari pengguna.

- Aplikasi harus mampu menerima input berikut dari pengguna *CSS selector* yang menjadi kriteria pencarian dari pengguna
- Aplikasi harus mampu menerima input berikut dari pengguna Pilihan jumlah hasil yang ingin ditampilkan (Top n kemunculan atau Semua kemunculan) dari pengguna.

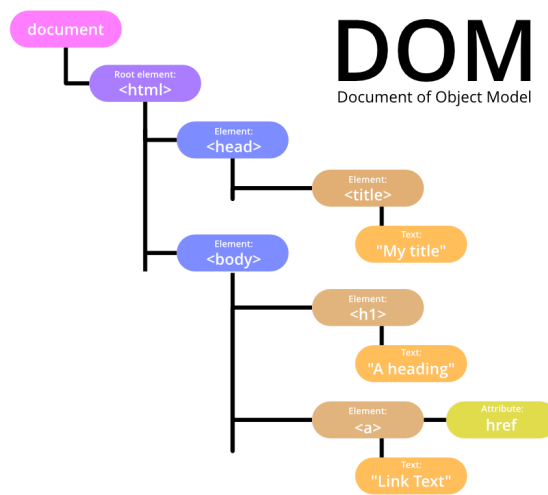
1.3.3 Fungsionalitas output

- Aplikasi harus menghasilkan output berupa Visualisasi struktur pohon DOM, dilengkapi dengan keterangan kedalaman maksimum *tree*.
- Aplikasi harus menghasilkan output berupa *highlight* jalur yang dilalui oleh algoritma penelusuran yang dipilih.
- Aplikasi harus menghasilkan output berupa Informasi waktu pencarian serta jumlah *node* yang dikunjungi.
- Aplikasi harus menghasilkan output berupa Penyimpanan *traversal log* yang mencatat tahapan penelusuran yang telah dilakukan.

BAB 2

LANDASAN TEORI

2.1 Document Object Model (DOM) dan Struktur HTML



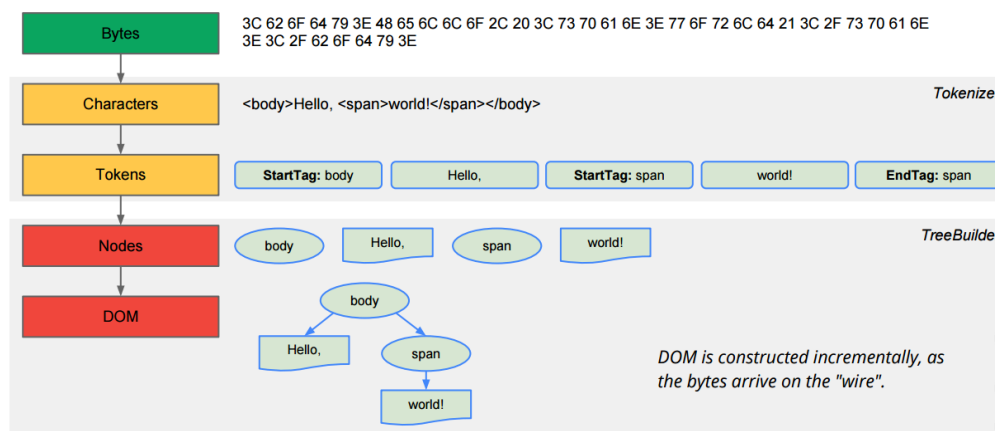
Gambar 1. Struktur Pohon DOM

<https://www.geeksforgeeks.org/java/what-is-document-object-in-java-dom/>

Document Object Model (DOM) merupakan sebuah antarmuka pemrograman aplikasi (API) yang merepresentasikan dokumen HTML secara terstruktur dalam bentuk pohon (*tree*), yang memungkinkan setiap elemen di dalam halaman web dapat diakses, dimodifikasi, dan dimanipulasi melalui logika pemrograman [1]. Secara teoritis dalam ilmu informatika, struktur DOM dipandang sebagai representasi konkret dari hubungan antar elemen HTML yang mencakup relasi orang tua (*parent*), anak (*child*), dan saudara (*sibling*) untuk memudahkan pengolahan dokumen secara sistematis, terstruktur, serta terprogram. Struktur ini secara fundamental selalu diawali oleh elemen akar (*root*) berupa tag `<html>` yang kemudian terbagi menjadi dua segmen utama, yaitu bagian `<head>` yang bertugas menampung metadata seperti judul halaman, tautan *stylesheet*, dan skrip, serta bagian `<body>` yang menjadi wadah bagi seluruh konten utama seperti teks, gambar, tombol, dan elemen lainnya. Secara sistematis, pohon DOM dapat dimodelkan sebagai graf berakar yang tidak memiliki siklus atau *Directed Acyclic Graph* (DAG) dan dinotasikan dalam himpunan $G = (V, E)$, V merupakan sekumpulan simpul (*nodes*) yang merepresentasikan objek elemen, dan E merupakan himpunan busur yang merepresentasikan keterkaitan

hierarkis antar elemen tersebut [2]. Pendekatan ini memastikan bahwa setiap perubahan pada satu simpul dapat diprediksi pengaruhnya terhadap simpul turunan dibawahnya, serta menciptakan kerangka kerja yang stabil untuk manipulasi dokumen web secara dinamis.

2.2 Mekanisme Leksikal dan Sintaksis HTML Parsing

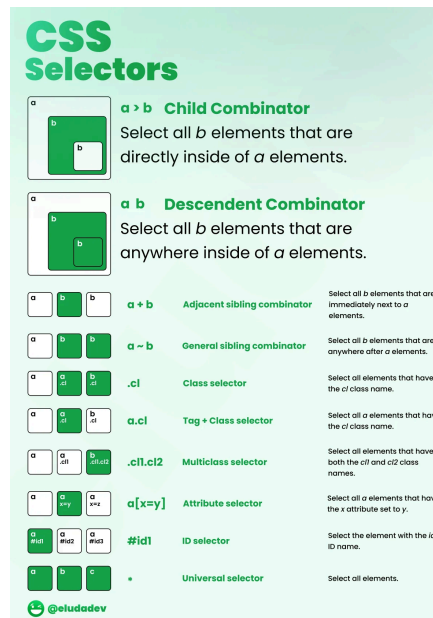


Gambar 2. Proses Parsing Pohon HTML

<https://stackoverflow.com/questions/3150293/how-does-a-parser-for-example-html-work>

Mekanisme leksikal adalah proses transformasi dokumen dari sekadar aliran data menjadi struktur pohon yang kompleks diawali melalui tahapan HTML Parsing, yakni proses membaca dokumen secara berurutan untuk mengubahnya menjadi struktur data pohon yang merepresentasikan hubungan antar elemen. Tahapan ini dimulai dari tingkat paling rendah, aliran *bytes* dikonversi menjadi karakter, kemudian diproses oleh sebuah *tokenizer* untuk mengidentifikasi unit-unit bermakna yang disebut token. Token-token tersebut mencakup *StartTag*, *EndTag*, komentar, hingga konten teks mentah yang nantinya akan mengisi simpul-simpul pada pohon. Selama proses pembangunan pohon, parser akan memproses token-token tersebut secara berurutan dan menerapkan aturan bahwa *tag* yang berada di dalam lingkup *tag* lain secara otomatis akan menjadi *child node*, sementara *tag* yang membungkusnya bertindak sebagai *parent node*. Dinamika ini memastikan bahwa setiap elemen memiliki tempat yang presisi dalam hierarki DOM, sehingga hasil akhirnya adalah sebuah DOM *Tree* utuh dan siap untuk ditelusuri lebih lanjut.

2.3 Pola Pencocokan CSS Selector dan Operasi Combinator

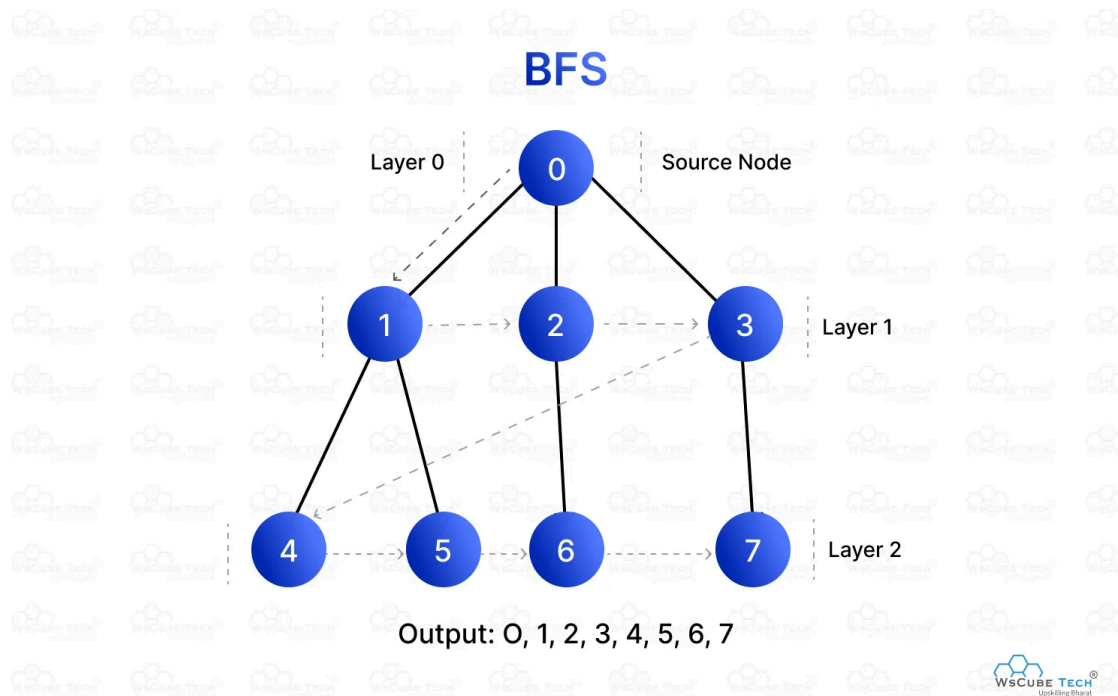


Gambar 3. Visualisasi CSS Selector

https://www.reddit.com/r/webdev/comments/ur6v5m/css_selectors_visually_explained/

CSS Selector adalah mekanisme pada CSS yang berfungsi sebagai pola pencocokan elemen HTML pada struktur DOM supaya dapat diberikan aturan visual atau dimanipulasi oleh algoritma. Terdapat beberapa kategori *selector* dasar yang sangat umum digunakan, yaitu *Tag Selector* yang memilih elemen berdasarkan nama identitas tag-nya (simbol <p>), *Class Selector* yang menggunakan atribut kelas dengan notasi titik (.), *Universal Selector* yang memilih semua elemen HTML, serta *ID Selector* yang menargetkan elemen unik dengan notasi pagar (#). Selain *selector* tunggal, terdapat mekanisme yang lebih kompleks yang disebut *Combinator*, yang merupakan metode untuk menggabungkan beberapa jenis *selector* untuk menentukan elemen yang terpengaruh berdasarkan hubungannya dengan elemen lain di dalam pohon. Hal ini mencakup *Child Combinator* (simbol >) yang hanya memilih anak langsung, serta *Descendant Combinator* (spasi) yang memilih seluruh elemen keturunan di level kedalaman manapun di bawah elemen referensi. Selain itu, terdapat pula *Adjacent Sibling Combinator* (Simbol +) dan *General Sibling Combinator* (simbol ~) yang digunakan untuk melakukan penelusuran pada simpul-simpul yang berada pada level horizontal yang sama.

2.4 Strategi Penelusuran Breadth-First Search (BFS)

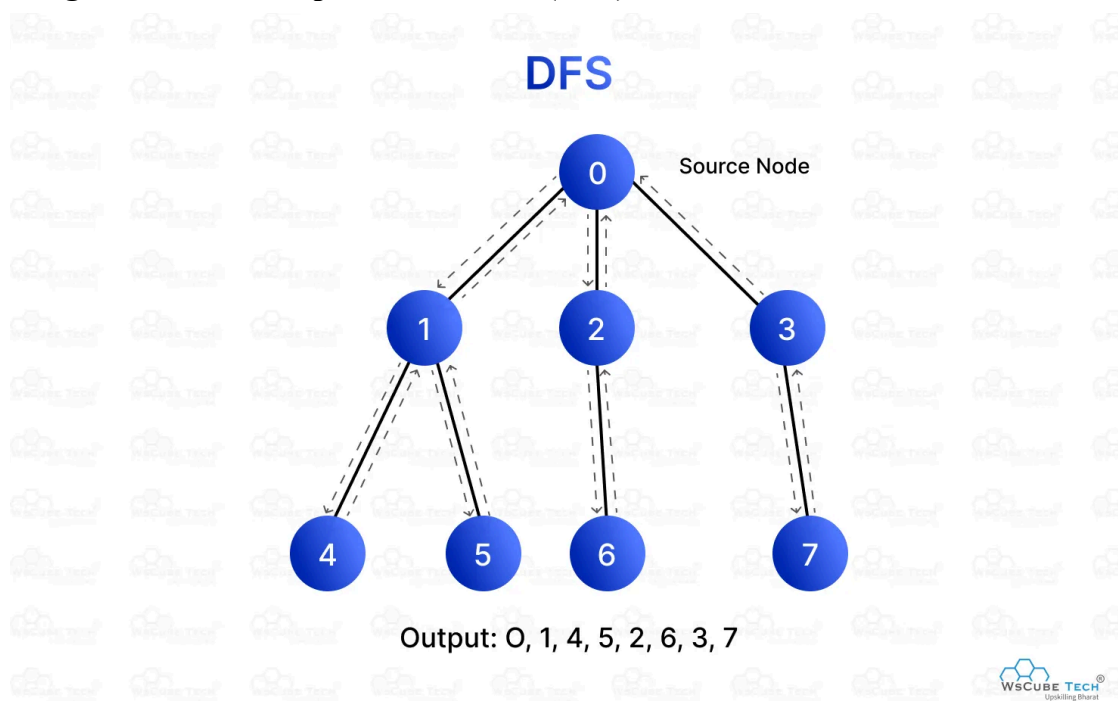


Gambar 4. Visualisasi BFS

<https://www.wscubetech.com/resources/dsa/dfs-vs-bfs>

Dalam konteks penelusuran pohon DOM pada tugas ini, salah satu strategi algoritma yang digunakan adalah *Breadth-First Search* (BFS) yang melakukan eksplorasi simpul secara melebar. Algoritma ini bekerja dengan mengunjungi simpul *root* terlebih dahulu, kemudian secara sistematis menjelajahi seluruh simpul anak pada tingkat kedalaman k sebelum berpindah ke tingkat kedalaman $k + 1$. BFS sangat bergantung pada struktur data antrian (*Queue*) yang mengikuti prinsip *First-In-First-Out* (FIFO) untuk mengelola urutan simpul yang akan dikunjungi. Secara analitis, kompleksitas waktu dari algoritma ini adalah $O(V + E)$ di mana V adalah jumlah simpul dan E adalah jumlah busur, karena setiap elemen dalam pohon DOM akan diperiksa setidaknya satu kali untuk mencocokkan kriterianya dengan *CSS Selector* yang diberikan. Keunggulan utama BFS dalam penelusuran DOM adalah kemampuannya untuk menemukan elemen yang terletak paling dekat dengan *root* dalam waktu yang relatif cepat serta kemudahan dalam memvisualisasikan jalur penelusuran per level kedalaman.

2.5 Strategi Penelusuran Depth-First Search (DFS)



Gambar 5. Visualisasi BFS

<https://www.wscubetech.com/resources/dsa/dfs-vs-bfs>

Sebagai alternatif dari pendekatan melebar, strategi *Depth-First Search* (DFS) diimplementasikan untuk melakukan penelusuran pohon secara mendalam dengan menyelami satu cabang hingga mencapai titik terjauh sebelum melakukan proses mundur atau *backtracking*. Algoritma ini biasanya memanfaatkan struktur data tumpukan (*Stack*) atau melalui pemanggilan fungsi secara rekursif untuk mengeksplorasi anak-anak dari sebuah simpul sesegera mungkin. Dalam penelusuran DOM, DFS akan masuk ke cabang pertama yang ditemui (biasanya anak paling kiri) dan terus menelusuri keturunannya hingga mencapai *leaf node* atau simpul daun, baru kemudian kembali untuk mengecek cabang saudara lainnya. Sama halnya dengan BFS, DFS memiliki efisiensi waktu sebesar $O(V + E)$, tetapi memiliki karakteristik penggunaan memori yang berbeda tergantung pada kedalaman maksimum dari pohon yang diproses. Penggunaan DFS sangat relevan dalam pencarian elemen yang berada di dalam struktur bersarang yang sangat dalam, dan algoritma ini akan memberikan hasil penelusuran yang mencerminkan urutan penulisan elemen di dalam dokumen HTML mentah.

BAB 3

APLIKASI BFS DAN DFS PADA PENELUSURAN POHON DOM

3.1 Representasi DOM sebagai Struktur Pohon (Tree)

Dokumen HTML pada dasarnya memiliki struktur hierarkis. Agar algoritma pencarian dapat diterapkan, dokumen HTML mentah pertama-tama diurai (*parsing*) dan direpresentasikan ke dalam struktur data pohon (N-ary tree). Dalam sistem ini, setiap elemen HTML direpresentasikan sebagai sebuah simpul (node) yang saling terhubung. Setiap node menyimpan informasi penting seperti:

- Tag: Nama elemen HTML (misalnya: `div`, `p`, `a`).
- Atribut: Kumpulan properti dari elemen tersebut seperti `id`, `class`, atau `href`.
- Konten (Content): Teks murni yang berada di dalam elemen (teks node).
- Relasi Struktural: Setiap node memiliki tautan (pointer) ke node induk (*Parent*) dan daftar node anak (*Children*), yang secara kolektif membentuk struktur pohon yang berakar pada node html.

Dengan representasi pohon N-ary ini, pencarian elemen HTML secara spesifik (seperti CSS Selector) direduksi menjadi masalah graf, di mana algoritma penelusuran standar seperti BFS dan DFS dapat digunakan untuk mengunjungi dan mengevaluasi setiap simpul.

3.2 Penelusuran Menggunakan Breadth-First Search (BFS)

Algoritma *Breadth-First Search* (BFS) melakukan penelusuran secara melebar. Pada pohon DOM, ini berarti algoritma akan menyapu dan memeriksa elemen "lantai demi lantai" atau berdasarkan tingkat kedalamannya (level-order).

Mekanisme:

1. Inisialisasi Antrean (Queue)

BFS menggunakan struktur data *queue* (antrean). Penelusuran dimulai dengan memasukkan *root* (node teratas) ke dalam antrean.

2. Evaluasi Simpul

Selama antrean tidak kosong, simpul pada posisi terdepan dikeluarkan. Sistem kemudian mencatat simpul tersebut sebagai "dikunjungi" dan mengevaluasinya terhadap fungsi kriteria (apakah simpul ini cocok dengan CSS Selector yang dicari).

3. Ekspansi

Semua node anak (*children*) dari simpul yang sedang dievaluasi akan dimasukkan ke bagian belakang antrean.

4. Penghentian Dini

Untuk efisiensi, algoritma mendukung batasan jumlah (*limit*). Jika jumlah elemen yang cocok telah memenuhi *limit* yang diminta, proses penelusuran akan langsung dihentikan tanpa harus mengelilingi seluruh sisa pohon.

Pendekatan BFS sangat optimal jika elemen HTML yang dicari berada di bagian luar atau tidak terlalu dalam dari struktur dokumen.

3.3 Paralelisasi Breadth-First Search (Parallel BFS)

Untuk mempercepat proses pencarian pada pohon DOM yang sangat besar dan kompleks, sistem ini juga mengaplikasikan varian BFS yang berjalan secara paralel menggunakan konsep *Worker Pool*. Berbeda dengan BFS sekuensial yang menggunakan antrian tunggal, *Parallel BFS* memproses simpul-simpul dalam satu level (tingkat kedalaman yang sama) secara serentak.

Mekanisme:

1. Pohon dieksplorasi per level (*currentLevel*).
2. Sejumlah goroutine (*workers*) ditugaskan untuk melakukan evaluasi kecocokan (*matching*) terhadap simpul-simpul di level tersebut secara bersamaan.
3. Setelah seluruh simpul di level tersebut selesai dievaluasi, sistem mengumpulkan semua anak dari simpul-simpul tersebut untuk menjadi *nextLevel*, dan proses diulang hingga elemen ditemukan atau pohon habis.

3.4 Paralelisasi Menggunakan Depth-First Search (DFS)

Berkebalikan dengan BFS, Depth-First Search (DFS) melakukan penelusuran secara mendalam. DFS akan masuk ke cabang anak terdalam terlebih dahulu sebelum beralih ke simpul saudara (*sibling*).

Mekanisme:

1. Inisialisasi Tumpukan (*Stack*): DFS menggunakan struktur data *stack* (tumpukan) yang bekerja dengan prinsip Last-In-First-Out (LIFO).
2. Penanganan Urutan Anak: Agar penelusuran sesuai dengan urutan baca dokumen HTML (dari kiri ke kanan), anak-anak dari sebuah simpul dimasukkan ke dalam *stack* secara terbalik (dari anak terakhir hingga anak pertama). Dengan demikian, anak pertama akan berada di puncak *stack* dan diproses lebih dulu.
3. Evaluasi dan Ekspansi: Sama seperti BFS, simpul dikeluarkan dari puncak *stack*, dievaluasi apakah cocok dengan kriteria selektor, lalu anak-anaknya di-push ke dalam *stack*.

DFS unggul ketika elemen yang dicari bersarang sangat dalam di dalam struktur dokumen HTML (nested elements), karena ia akan langsung menukik ke dasar pohon DOM.

3.5 Aplikasi Selektor dalam Penelusuran

Baik BFS maupun DFS bertugas murni sebagai "mesin penggerak" yang menjelajahi topologi dokumen. Agar dapat menemukan elemen spesifik (misalnya pencarian elemen dengan struktur `div > p.target`), sistem menggunakan arsitektur pemisahan tugas sebagai berikut:

- String pencarian dari pengguna diurai (di-*parse*) menjadi serangkaian langkah kombinator (*Descendant*, *Child*, *Sibling*) dan selektor sederhana (*Tag*, *ID*, *Class*).
- Langkah-langkah tersebut dikompilasi menjadi sebuah Fungsi Kriteria (*MatchFunc*).
- Algoritma BFS/DFS kemudian memanggil *MatchFunc* ini pada setiap simpul yang sedang disinggahi. Jika fungsi mengembalikan nilai `true`, simpul tersebut dimasukkan ke dalam daftar hasil tangkapan algoritma.

BAB 4

IMPLEMENTASI DAN PENGUJIAN

4.1 Implementasi

4.1.1 Implementasi Pengambil Dokumen (Scraper) dan DOM Builder

Scraper bertugas mengambil dokumen HTML menggunakan modul net/http. Untuk menghindari program menggantung (*hanging*) akibat koneksi buruk, diimplementasikan batasan waktu (*timeout*) sebesar 15 detik.

```
func FetchHTML(url string) (string, error) {
    client := &http.Client{
        Timeout: 15 * time.Second,
    }

    resp, err := client.Get(url)
    if err != nil {
        return "", fmt.Errorf("kurir gagal mencapai alamat %s: %w",
url, err)
    }

    defer resp.Body.Close()

    if resp.StatusCode != http.StatusOK {
        return "", fmt.Errorf("server menolak permintaan (Status
HTTP: %d)", resp.StatusCode)
    }

    htmlBytes, err := io.ReadAll(resp.Body)
    if err != nil {
        return "", fmt.Errorf("berhasil menghubungi server, tapi
gagal membaca isi dokumen HTML: %w", err)
    }

    return string(htmlBytes), nil
}
```

Setelah dokumen HTML didapatkan, *parser* akan membaca teks dan membentuk hierarki pohon DOM. Setiap elemen direpresentasikan dengan struktur data Node yang saling menautkan *Parent* dan sekumpulan *Children*.

```
type Node struct {
    Tag          string
    Attributes   map[string]string
    Content      string
    Children     []*Node
    Parent       *Node
}
```

4.1.2 Implementasi Engine Selektor CSS

Input query dari pengguna diproses oleh modul selektor. Proses ini memecah string menjadi logika kombinator (*Descendant*, *Child*, *Adjacent Sibling*, *General Sibling*) dan menerjemahkannya menjadi aturan evaluasi (*matcher*).

```
type combinator int

const (
    combinatorNone combinator = iota
    combinatorDescendant
    combinatorChild
    combinatorAdjacentSibling
    combinatorGeneralSibling
)
```

Aturan ini dibungkus menjadi sebuah fungsi bertipe MatchFunc (fungsi predikat: func(*dom.Node) bool). Fungsi ini dieksekusi pada setiap *Node* yang dilewati oleh algoritma penelusuran. Jika sesuai, fungsi mengembalikan nilai true.

4.1.3 Implementasi BFS, DFS, dan Mode Paralel

Algoritma utama dijalankan secara terpisah berdasarkan metode yang dipilih pengguna.

4.1.3.1 BFS

BFS diimplementasikan menggunakan tipe data *slice* sebagai *queue* (antrean). Simpul dievaluasi secara horizontal berdasar level. Anak-anak dari simpul yang sedang dievaluasi selalu ditambahkan di akhir antrean.

```
func BFS(root *dom.Node, isMatch MatchFunc, limit int) Result {
    start := time.Now()
    result := initResult()

    if root == nil {
        return result
    }

    queue := []*dom.Node{root}
    foundEnough := false

    for len(queue) > 0 && !foundEnough {
        current := queue[0]
        queue = queue[1:]

        recordVisit(&result, current)

        if isMatch == nil || isMatch(current) {
            result.Matches = append(result.Matches,
```

```
current)
    if limit > 0 && len(result.Matches) >= limit
    {
        foundEnough = true
        break
    }
    queue = append(queue, current.Children...)
}
result.Duration = time.Since(start)
return result
}
```

4.1.3.2 DFS

DFS diimplementasikan menggunakan tipe data *slice* sebagai *stack* (tumpukan) dengan prinsip *Last-In-First-Out*. Agar penelusuran sesuai urutan baca, anak-anak ditambahkan ke *stack* dari urutan indeks terakhir hingga pertama (i--).

```
func DFS(root *dom.Node, isMatch MatchFunc, limit int) Result {
    start := time.Now()
    result := initResult()

    if root == nil {
        return result
    }

    stack := []*dom.Node{root}
    foundEnough := false

    for len(stack) > 0 && !foundEnough {
        lastIdx := len(stack) - 1
        current := stack[lastIdx]
        stack = stack[:lastIdx]

        recordVisit(&result, current)

        if isMatch == nil || isMatch(current) {
            result.Matches = append(result.Matches,
current)
            if limit > 0 && len(result.Matches) >= limit
            {
                foundEnough = true
                break
            }
        }
    }
}
```

```
        for i := len(current.Children) - 1; i >= 0; i-- {
            stack = append(stack, current.Children[i])
        }
    }

    result.Duration = time.Since(start)
    return result
}
```

4.1.3.3 BFS (Pararel)

Untuk pohon DOM yang sangat besar, evaluasi simpul di level yang sama dapat dilakukan bersamaan menggunakan Goroutine. Evaluasi dilakukan serentak, dan hasilnya diamankan ke dalam matchFlags untuk menghindari tumpang tindih memori (*race condition*).

```
currentLevel := []*dom.Node{root}
stopSearching := false

for len(currentLevel) > 0 && !stopSearching {
    matchFlags := make([]bool, len(currentLevel))

    parallelFor(len(currentLevel), workers, func(i int)
    {
        node := currentLevel[i]
        if isMatch == nil || isMatch(node) {
            matchFlags[i] = true
        }
    })

    // Pengumpulan hasil dan pembentukan nextLevel
}
```

4.2 Pengujian

4.2.1 Test Cast 1 (Demo BFS)

Data Input/Output

SehatSehat.KuatKuat

IF2211 Strategi Algoritma - BFS & DFS & LCA

ITB - 2025/2026

BFS / DFS

LCA

Demo Node

KONFIGURASI TRAVERSAL

SUMBER HTML

URL Website

Raw HTML

URL

https://example.com

CSS SELECTOR

p

Contoh: p, .class, #id, div > span

ALGORITMA TRAVERSAL

BFS

BREADTH FIRST

DFS

DEPTH FIRST

JUMLAH HASIL

Semua

Top N

STATISTIK

algo: BFS selector: p

4 ms

Waktu Pencarian

13

Node Dikunjungi

13

Total Node

5

Kedalaman Maks

2

Hasil Ditemukan

ANIMASI TRAVERSAL

0.5x

1x

2x

5x

Ulang

Langkah 13 / 13 ✓ Selesai

Pohon DOM

Hasil (2)

Log

☐ Sedang dikunjungi ☒ Match selector ☐ Belum dikunjungi

algo: BFS

```
<html>
  <head>
    <title>
    <meta>
  <body>
    <div class="container">
      <h1>
      <p>
        <span class="highlight">
      <p class="info">
      <nav>
        <a>
        <a class="info">
```

13 node total · kedalaman maks 4

Pohon DOM

Hasil (2)

Log

#1 root.1.0.1 depth: 3
<p> This domain is for illustrative examples.</p>
html > body > div.container > p

#2 root.1.0.2 depth: 3
<p class="info"> You may use this domain in examples.</p>
html > body > div.container > p.info

Pohon DOM

Hasil (2)

Log

13 langkah · 2 match [Download Log](#)

#	UID	Label	Match
1	root	<html>	—
2	root.0	<head>	—
3	root.1	<body>	—
4	root.0.0	<title>	—
5	root.0.1	<meta>	—
6	root.1.0	<div.container>	—
7	root.1.0.0	<h1>	—
8	root.1.0.1	<p>	✓

4.2.2 Test Cast 2 (Demo DFS)

Data Input/Output

SehatSehat.KuatKuat

IF2211 Strategi Algoritma - BFS & DFS & LCA

ITB - 2025/2026

BFS / DFS

LCA

Demo Node

X

KONFIGURASI TRAVERSAL

SUMBER HTML

URL Website

Raw HTML

URL

https://example.com

CSS SELECTOR

p

Contoh: p, .class, #id, div > span

ALGORITMA TRAVERSAL

BFS

BREADTH FIRST

DFS

DEPTH FIRST

JUMLAH HASIL

Semua

Top N

STATISTIK

algo: DFS selector: p

2 ms

Waktu Pencarian

13

Node Dikunjungi

13

Total Node

5

Kedalaman Maks

2

Hasil Ditemukan

ANIMASI TRAVERSAL

0.5x 1x 2x 5x

Ulang

Langkah 13 / 13 Selesai

Pohon DOM

Hasil (2)

Log

☐ Sedang dikunjungi

☒ Match selector

☐ Belum dikunjungi

algo: DFS

Pohon DOM

Hasil (2)

Log

☐ Sedang dikunjungi

☒ Match selector

☐ Belum dikunjungi

algo: DFS

```
<html>
  <head>
    <title>
    <meta>
  <body>
    <div class="container">
      <h1>
      <p>
        <span class="highlight">
        <p class="info">
      <nav>
        <a>
        <a class="info">
```

13 node total · kedalaman maks 4

Halaman 19

Pohon DOM

Hasil (2)

Log

#1 root.1.0.1 depth: 3

This domain is for illustrative examples.</p>

html > body > div.container > p

#2 root.1.0.2 depth: 3

html > body > div.container > p.info

Pohon DOM

Hasil (2)

Log

13 langkah · 2 match [Download Log](#)

#	UID	Label	Match
1	root	<html>	—
2	root.0	<head>	—
3	root.0.0	<title>	—
4	root.0.1	<meta>	—
5	root.1	<body>	—
6	root.1.0	<div.container>	—
7	root.1.0.0	<h1>	—
8	root.1.0.1	<p>	✓

4.2.3 Test Case 3 (Demo LCA)

Data Input/Output

SehatSehat.KuatKuat

IF2211 Strategi Algoritma · BFS & DFS & LCA

ITB · 2025/2026

BFS / DFS

LCA

KONFIGURASI LCA

Lowest Common Ancestor (LCA)
Mencari node leluhur terdekat yang sama dari dua elemen HTML yang dipilih via CSS selector.

SUMBER HTML

URL Website

Raw HTML

URL

https://example.com

NODE A - CSS SELECTOR

h1, .title, #header

NODE B - CSS SELECTOR

p, .content, #footer

HASIL LCA

VISUALISASI PENCARIAN LCA

LCA node Node A Node B Sedang dikunjungi

<html>

<body>

<div.container>

<h1>

<p>

<span.highlight>

Selesai

Kecepatan

0.5x 1x 2x 5x

✓ Selesai

Langkah 6 / 6 ✓ LCA ditemukan

LCA Node

root.1.0

depth: 2

<div class="container"> Example Domain This domain is for illustrative examples.</div>

html > body > div.container

4.2.4 Test Case 4 (BFS: URL)

Data Input

URL: <https://example.com>

CSS Selector: p

Algoritma Traversal: BFS

Jumlah Hasil: Semua

Data Output

The screenshot displays the 'SehatSehat.KuatKuat' application interface for the IF2211 Strategi Algoritma course. The interface is divided into several sections:

- KONFIGURASI TRAVERSAL:** Includes a 'SUMBER HTML' section with a 'URL Website' button and a 'Raw HTML' button. The 'URL' field contains 'https://example.com'. The 'CSS SELECTOR' field contains 'p'. The 'ALGORITMA TRAVERSAL' section has two buttons: 'BFS BREADTH FIRST' (selected) and 'DFS DEPTH FIRST'. The 'JUMLAH HASIL' section has two buttons: 'Semua' (selected) and 'Top N'.
- STATISTIK:** Displays the following statistics:
 - algo: BFS selector: p
 - < 1 ms Waktu Pencarian
 - 16 Node Dikunjungi
 - 16 Total Node
 - 6 Kedalaman Maks
 - 2 Hasil Ditemukan
- ANIMASI TRAVERSAL:** Includes a progress bar and a 'Ulang' button. The progress bar shows 'Langkah 16 / 16' and 'Selesai'.
- Pohon DOM:** Displays the DOM tree structure. The 'Hasil (2)' section shows the results of the traversal, including the 'Match selector' and 'Belum dikunjungi' status. The 'Log' section shows the traversal steps.

Pohon DOM

Hasil (2)

Log

☐ Sedang dikunjungi ☒ Match selector ☐ Belum dikunjungi

algo: BFS

```
<html>
  <head>
    <title>
      <#text>
    <meta>
    <style>
      <#text>
  <body>
    <div>
      <h1>
        <#text>
      <p>
        <#text>
      <p>
        <a>
          <#text>
```

16 node total · kedalaman maks 5

#1 root.1.0.1 depth: 3

<p> This domain is for use in documentation examples without needing permission. Avo...</p>

html > body > div > p

#2 root.1.0.2 depth: 3

<p> Learn more</p>

html > body > div > p

Pohon DOM

Hasil (2)

Log

16 langkah · 2 match

[Download Log](#)

#	UID	Label	Match
9	root.0.2.0	<#text>	—
10	root.1.0.0	<h1>	—
11	root.1.0.1	<p>	✓
12	root.1.0.2	<p>	✓
13	root.1.0.0.0	<#text>	—
14	root.1.0.1.0	<#text>	—
15	root.1.0.2.0	<a>	—
16	root.1.0.2.0.0	<#text>	—

4.2.5 Test Case 5 (DFS: URL)

Data Input

URL: <https://example.com>

CSS Selector: div

Algoritma Traversal: DFS

Jumlah Hasil: Semua

Data Output

The screenshot displays the 'SehatSehat.KuatKuat' web application interface. The top navigation bar includes the title 'SehatSehat.KuatKuat', the course 'IF2211 Strategi Algoritma - BFS & DFS & LCA', and the date 'ITB - 2025/2026'. The main interface is divided into several sections:

- KONFIGURASI TRAVERSAL:** Includes a 'SUMBER HTML' section with a 'URL Website' button and a 'Raw HTML' button. The 'URL' field contains 'https://example.com'. The 'CSS SELECTOR' field contains 'div'. The 'ALGORITMA TRAVERSAL' section has buttons for 'BFS' and 'DFS' (selected). The 'JUMLAH HASIL' section has buttons for 'Semua' (selected) and 'Top N'.
- STATISTIK:** Displays search statistics: 'algo: DFS selector: div', 'Waktu Pencarian: < 1 ms', 'Node Dikunjungi: 16', 'Total Node: 16', 'Kedalaman Maks: 6', and 'Hasil Ditemukan: 1'.
- ANIMASI TRAVERSAL:** Features a progress bar and a 'Ulang' button. The status shows 'Langkah 16 / 16' and 'Selesai'.
- Pohon DOM:** Displays the DOM tree structure, showing the root node 'html' and its children 'head' and 'body'.

Pohon DOM

Hasil (1)

Log

☐ Sedang dikunjungi ☒ Match selector ☐ Belum dikunjungi

algo: DFS

```
<html>
  <head>
    <title>
      <#text>
    <meta>
    <style>
      <#text>
  <body>
    <div>
      <h1>
        <#text>
      <p>
        <#text>
      <p>
        <a>
          <#text>
```

16 node total · kedalaman maks 5

Pohon DOM

Hasil (1)

Log

```
#1 root.1.0 depth: 2
<div> Example Domain This domain is for use in documentation examples without needing ...
</div>
html > body > div
```

Pohon DOM

Hasil (1)

Log

16 langkah · 1 match [Download Log](#)

4.2.6 Test Case 6 (LCA: URL)

Data Input

URL: <https://example.com>

Node A: h1

Node B: p

Algoritma Traversal: LCA

Data Output

SehatSehat.KuatKuat
IF2211 Strategi Algoritma - BFS & DFS & LCA
ITB - 2025/2026

BFS / DFS **LCA**

KONFIGURASI LCA

Lowest Common Ancestor (LCA)
Mencari node leluhur terdekat yang sama dari dua elemen HTML yang dipilih via CSS selector.

SUMBER HTML

URL Website **Raw HTML**

URL

NODE A - CSS SELECTOR

NODE B - CSS SELECTOR

HASIL LCA

VISUALISASI PENCARIAN LCA

☐ LCA node ☐ Node A ☐ Node B ☐ Sedang dikunjungi

`<html>`
└─ `<body>`
 └─ `<div>` LCA
 └─ `<h1>` A
 └─ `<p>` B

Selesai

Kecepatan

✓ Selesai

Langkah 5 / 5 ✓ LCA ditemukan

LCA Node root.1.0 depth: 2
`<div> Example Domain This domain is for use in documentation examp...</div>`
└─ html > body > div

LCA Node root.1.0 depth: 2
`<div> Example Domain This domain is for use in documentation examp...</div>`
└─ html > body > div

Node A root.1.0.0 depth: 3
`<h1> Example Domain</h1>`
└─ html > body > div > h1

Node B root.1.0.1 depth: 3
`<p> This domain is for use in documentation examples without nee...</p>`
└─ html > body > div > p

4.2.7 Test Case 7 (BFS: RAW HTML)

Data Input
RAW HTML: <pre><!doctype html> <html lang="en"> <head> <meta charset="UTF-8" /> <meta name="viewport" content="width=device-width, initial-scale=1.0" /> <title>E-Library</title> </head> <body> <div id="root"></div> <script type="module" src="/src/main.jsx"></script> </body> </html></pre> CSS Selector: script Algoritma Traversal: BFS Jumlah Hasil: Semua
Data Output

SehatSehat.KuatKuat

IF2211 Strategi Algoritma · BFS & DFS & LCA

ITB · 2025/2026

BFS / DFS

LCA

KONFIGURASI TRAVERSAL

SUMBER HTML

URL Website

Raw HTML

HTML

```
<!doctype html>
<html>
  <head>
    <title>E-Library</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module"
      src="/src/main.jsx"></script>
  </body>
</html>
```

CSS SELECTOR

script

Contoh: p, .class, #id, div > span

ALGORITMA TRAVERSAL

BFS

DFS

STATISTIK

algo: BFS selector: script

< 1 ms

9

9

Waktu Pencarian

Node Dikunjungi

Total Node

4

1

Kedalaman Maks

Hasil Ditemukan

ANIMASI TRAVERSAL

0.5x 1x 2x 5x

Ulang

Langkah 9 / 9 ✓ Selesai

Pohon DOM

Hasil (1)

Log

☐ Sedang dikunjungi ☒ Match selector ☐ Belum dikunjungi

algo: BFS

```
<html>
  <head>
```

Pohon DOM

Hasil (1)

Log

☐ Sedang dikunjungi ☒ Match selector ☐ Belum dikunjungi

algo: BFS

```
<html>
  <head>
    <meta>
    <meta>
    <title>
      <#text>
    </title>
  <body>
    <div#root>
      <script>
```

MATCH

9 node total · kedalaman maks 3

Pohon DOM

Hasil (1)

Log

```
#1 root.1.1
<script></script>
html > body > script
```

depth: 2

Pohon DOM		Hasil (1)	Log
9 Langkah · 1 match		Download Log	
#	UID	Label	Match
2	root.0	<head>	—
3	root.1	<body>	—
4	root.0.0	<meta>	—
5	root.0.1	<meta>	—
6	root.0.2	<title>	—
7	root.1.0	<div#root>	—
8	root.1.1	<script>	✓
9	root.0.2.0	<#text>	—

4.2.8 Test Case 8 (DFS: RAW HTML)

Data Input
RAW HTML: <pre><!doctype html> <html lang="en"> <head> <meta charset="UTF-8" /> <meta name="viewport" content="width=device-width, initial-scale=1.0" /> <title>E-Library</title> </head> <body> <div id="root"></div> <script type="module" src="/src/main.jsx"></script> </body></pre>

```
</html>
```

CSS Selector: meta

Algoritma Traversal: DFS

Jumlah Hasil: Semua

Data Output

SehatSehat. KuatKuat
IF2211 Strategi Algoritma - BFS & DFS & LCA
ITB - 2025/2026

BFS / DFS LCA

KONFIGURASI TRAVERSAL

SUMBER HTML

URL Website Raw HTML

HTML

```
<html>
<head>
  <meta charset="UTF-8" />
  <meta name="viewport"
    content="width=device-width,
    initial-scale=1.0" />
  <title>E-Library</title>
</head>
<body>
  <div id="root"></div>
</body>
</html>
```

CSS SELECTOR

meta

Contoh: p, .class, #id, div > span

ALGORITMA TRAVERSAL

BFS DFS

STATISTIK

algo: DFS selector: meta

< 1 ms Waktu Pencarian	9 Node Dikunjungi	9 Total Node
4 Kedalaman Maks	2 Hasil Ditemukan	

ANIMASI TRAVERSAL

0.5x 1x 2x 5x

Ulang

Langkah 9 / 9 ✓ Selesai

Pohon DOM Hasil (2) Log

☐ Sedang dikunjungi ☒ Match selector ☐ Belum dikunjungi algo: DFS

```
<html>
  <head>
    <meta> MATCH
    <meta> MATCH
    <title>
      <#text>
    </title>
  </head>
  <body>
    <div#root>
    </div>
    <script>
    </script>
  </body>
</html>
```

Pohon DOM Hasil (2) Log

☐ Sedang dikunjungi ☒ Match selector ☐ Belum dikunjungi algo: DFS

```
<html>
  <head>
    <meta> MATCH
    <meta> MATCH
    <title>
      <#text>
    </title>
  </head>
  <body>
    <div#root>
    </div>
    <script>
    </script>
  </body>
</html>
```

9 node total · kedalaman maks 3

Pohon DOM

Hasil (2)

Log

#1 root.0.0 depth: 2

<meta></meta>

html > head > meta

#2 root.0.1 depth: 2

<meta></meta>

html > head > meta

Pohon DOM

Hasil (2)

Log

9 Langkah • 2 match [Download Log](#)

#	UID	Label	Match
1	root	<html>	—
2	root.0	<head>	—
3	root.0.0	<meta>	✓
4	root.0.1	<meta>	✓
5	root.0.2	<title>	—
6	root.0.2.0	<#text>	—
7	root.1	<body>	—
8	root.1.0	<div#root>	—

4.2.9 Test Case 9 (DFS: RAW HTML)

Data Input

RAW HTML:

```
<!doctype html>

<html lang="en">

  <head>

    <meta charset="UTF-8" />

    <meta name="viewport" content="width=device-width,
initial-scale=1.0" />

    <title>E-Library</title>
```

```
</head>

<body>

  <div id="root"></div>

  <script type="module" src="/src/main.jsx"></script>

</body>

</html>
```

Node A: title

Node B: meta

Algoritma Traversal: LCA

Data Output

The screenshot displays the 'SehatSehat.KuatKuat' LCA tool interface. The left sidebar contains the 'KONFIGURASI LCA' section with a description of the LCA algorithm and a 'SUMBER HTML' section where the HTML content is pasted. The main area shows the 'HASIL LCA' section, which includes a 'VISUALISASI PENCARIAN LCA' diagram and a 'Selesai' status bar. The diagram shows the HTML tree structure with the LCA node highlighted. The 'Selesai' bar indicates the search is complete, with a progress bar and a 'Kecepatan' (Speed) indicator. The 'Langkah 4 / 4' status is shown, indicating the LCA has been found.

KONFIGURASI LCA

Lowest Common Ancestor (LCA)
Mencari node leluhur terdekat yang sama dari dua elemen HTML yang dipilih via CSS selector.

SUMBER HTML
URL Website | Raw HTML

HTML

```
<?html>
<body>
  <div id="root"></div>
  <script type="module"
src="/src/main.jsx"></script>
</body>
</html>
```

NODE A - CSS SELECTOR
title

HASIL LCA

VISUALISASI PENCARIAN LCA

☒ LCA node ☒ Node A ☒ Node B ☐ Sedang dikunjungi

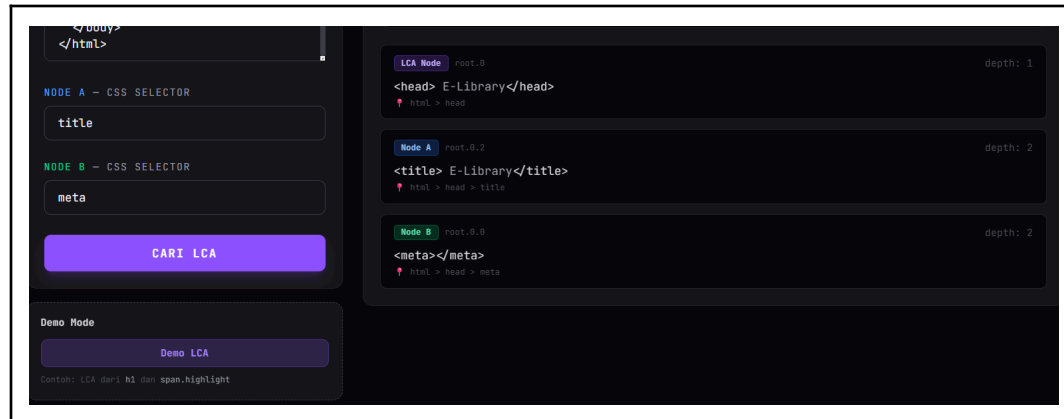
Selesai

Kecepatan: 0.5x 1x 2x 5x

Langkah 4 / 4 ✓ LCA ditemukan

LCA Node root.B depth: 1
<head> E-Library</head>

Node A root.B.2 depth: 2



4.2.10 Test Case 10 (Top N)

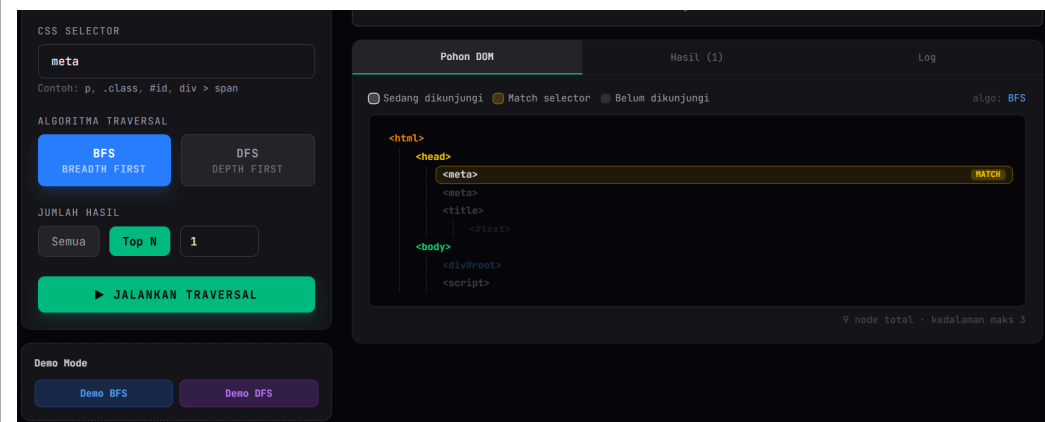
Data Input
RAW HTML: <pre><!doctype html> <html lang="en"> <head> <meta charset="UTF-8" /> <meta name="viewport" content="width=device-width, initial-scale=1.0" /> <title>E-Library</title> </head> <body> <div id="root"></div> <script type="module" src="/src/main.jsx"></script> </body> </html></pre>

CSS Selector:

Algoritma Traversal: BFS/DFS

Top N: 1

Data Output



4.2.11 Test Case 11 (Error Handling)

Data Input

RAW HTML:

```
<!doctype html>

<html lang="en">

  <head>

    <meta charset="UTF-8" />

    <meta name="viewport" content="width=device-width,
initial-scale=1.0" />

    <title>E-Library</title>
```

```
</head>

<body>

  <div id="root"></div>

  <script type="module" src="/src/main.jsx"></script>

</body>
</html>
```

CSS Selector:

Algoritma Traversal: BFS/DFS

Jumlah Hasil: Semua

Data Output

```
initial-scale=1.0" />
<title>E-Library</title>
</head>
<body>
  <div id="root"></div>
```

CSS SELECTOR

div.container > p, #header, .nav-link

Contoh: p, .class, #id, div > span

ALGORITMA TRAVERSAL

BFS
BREADTH FIRST

DFS
DEPTH FIRST

JUMLAH HASIL

Semua

Top N

⚠ CSS Selector tidak boleh kosong.

▶ JALANKAN TRAVERSAL

Demo Mode

Demo BFS

Demo DFS

4.2.12 Test Case 12 (Error Handling)

Data Input

RAW HTML:

CSS Selector: meta

Algoritma Traversal: BFS/DFS

Jumlah Hasil: Semua

Data Output

The screenshot shows a web application interface with a dark theme. It includes the following elements:

- CSS SELECTOR**: A text input field containing the value "meta". Below it, a hint text reads "Contoh: p, .class, #id, div > span".
- ALGORITMA TRAVERSAL**: Two buttons, "BFS BREADTH FIRST" (highlighted in blue) and "DFS DEPTH FIRST" (grey).
- JUMLAH HASIL**: Three buttons, "Semua" (grey), "Top N" (highlighted in green), and a numeric input field containing the value "1".
- Error Message**: A red-bordered box containing a warning icon and the text "HTML tidak boleh kosong." (HTML cannot be empty).
- Action Button**: A large green button with a right-pointing triangle icon and the text "JALANKAN TRAVERSAL".
- Demo Mode**: A section at the bottom with two buttons, "Demo BFS" (blue) and "Demo DFS" (purple).

4.2.13 Test Case 13 (Error Handling)

Data Input
RAW HTML: <pre><html> <head> <title>Test</title> </head> <body> <p>Hello World!</p> </body> </html></pre> CSS Selector: p Algoritma Traversal: BFS/DFS Jumlah Hasil: Semua

Data Output
CSS SELECTOR p Contoh: p, .class, #id, div > span ALGORITMA TRAVERSAL BFS BREADTH FIRST DFS DEPTH FIRST JUMLAH HASIL Semua Top N ⚠ Dokumen harus memiliki tag <html>. ▶ JALANKAN TRAVERSAL

4.2.13 Test Case 13 (Error Handling)

Data Input
RAW HTML: <pre><html> <head> <title>Test<</pre>

```
</head>

<body>

  <p>Hello World!

</body>

</html>
```

CSS Selector: p

Algoritma Traversal: BFS/DFS

Jumlah Hasil: Semua

Data Output

CSS SELECTOR

p

Contoh: p, .class, #id, div > span

ALGORITMA TRAVERSAL

BFS

BREADTH FIRST

DFS

DEPTH FIRST

JUMLAH HASIL

Semua

Top N

⚠ Struktur tag tidak valid: menutup
</head> tetapi yang diharapkan
</title>.

▶ **JALANKAN TRAVERSAL**

BAB 5

KESIMPULAN DAN SARAN

5.1 Kesimpulan

- 5.1.1 Tugas Besar ini berhasil mengimplementasikan sebuah aplikasi berbasis web untuk melakukan *traversal* pada struktur Document Object Model (DOM), yang merupakan representasi pohon dari dokumen HTML.
- 5.1.2 Aplikasi berhasil mengimplementasikan dan memanfaatkan dua algoritma penelusuran graf, yaitu Breadth First Search (BFS) dan Depth First Search (DFS), untuk mencari elemen pada pohon DOM yang sesuai dengan kriteria CSS Selector tertentu
- 5.1.3 Secara fungsionalitas, aplikasi telah memenuhi spesifikasi wajib dengan menyediakan fitur *web scraping*, pilihan algoritma traversal, penelusuran menggunakan *CSS selector*, visualisasi struktur pohon DOM, penandaan jalur penelusuran, serta menampilkan metrik kinerja berupa waktu pencarian dan jumlah *node* yang dikunjungi.
- 5.1.4 Perbandingan hasil pengujian menunjukkan bahwa kedua algoritma bekerja secara efektif dalam konteks penelusuran DOM, di mana BFS cenderung lebih cocok untuk penelusuran pada level-level awal (*breadth*), dan DFS cenderung lebih efisien untuk mencapai kedalaman tertentu (*depth*) dalam pohon

5.2 Saran

Untuk pengembangan aplikasi di masa mendatang, berikut adalah beberapa saran dari kami yang dapat dipertimbangkan:

- 5.2.1 Selain waktu dan jumlah *node* yang dikunjungi, disarankan untuk menambahkan fitur *profiling* yang lebih mendalam, misalnya dengan memvisualisasikan *cost* komputasi (jumlah perbandingan *selector* yang dilakukan) pada setiap level *node* untuk analisis efisiensi yang lebih akurat antara BFS dan DFS.
- 5.2.1 Sediakan antarmuka visual yang memungkinkan pengguna menjalankan BFS dan DFS secara bersamaan pada *CSS selector* yang sama, lalu menampilkan grafik perbandingan waktu eksekusi dan jumlah *node* yang dikunjungi. Ini akan mempermudah analisis empiris perbandingan efisiensi kedua algoritma.

LAMPIRAN

Berikut adalah pranala ke *repository* program:

<https://github.com/Dikanugraha-hub/SehatSehatKuatKuat>

Berikut adalah pranala ke video youtube:

https://youtu.be/MVVHnNJRyyg?si=SCV7gz3zyZvT_Jpo

Berikut adalah pranala ke website hasil deploy:

<http://20.187.66.64:3000/>

No	Poin	Ya	Tidak
1	Aplikasi berhasil di kompilasi tanpa kesalahan	✓	
2	Aplikasi berhasil dijalankan	✓	
3	Aplikasi dapat menerima input URL web, pilihan algoritma, CSS selector, dan jumlah hasil	✓	
4	Aplikasi dapat melakukan scraping terhadap web pada input	✓	
5	Aplikasi dapat menampilkan visualisasi pohon DOM	✓	
6	Aplikasi dapat menelusuri pohon DOM dan menampilkan hasil penelusuran	✓	
7	Aplikasi dapat menandai jalur tempuh oleh algoritma	✓	
8	Aplikasi dapat menyimpan jalur yang ditempuh algoritma dalam traversal log	✓	
9	[Bonus] Membuat video	✓	
10	[Bonus] Deploy aplikasi	✓	
11	[Bonus] Implementasi animasi pada penelusuran pohon	✓	
12	[Bonus] Implementasi multithreading	✓	
13	[Bonus] Implementasi LCA Binary Lifting	✓	

PEMBAGIAN TUGAS

No.	Nama	NIM	Tugas yang Dikerjakan
1	Jonathan Alveraldo Bangun	13524120	Implementasi DOM, lca, scraper, traversal (algoritma BFS dan DFS), membantu pembuatan laporan, mengedit video
2	Zahran Alvan Putra Winarko	13524124	Membuat implementasi untuk DOM CSS Traversal mulai dari: 1. Setup Project (Next.js, Typescript, Tailwind, Docker), 2. Integrasi API/endpoint dari backend, 3. Visualisasi pohon DOM, 4. Animasi traversal untuk masing-masing algoritma, 5. Komponen UI secara lengkap (Input form, statistik hasil, hasil match selektor), dan 6. Demo Mode.
3	Dika Pramudya Nugraha	13524132	Membuat parser (HTML Parsing), Membangun modul server sebagai entry point aplikasi backend yang mengimplementasikan HTTP Server, pengujian fungsionalitas dan error handling, menangani routing REST API, mengorkestrasi alur data dari penerimaan request JSON hingga pengembalian response ke antarmuka pengguna, serta membantu mengoptimalkan algoritma BFS dan DFS yang sudah rekan saya buat.

PERNYATAAN TIDAK MELAKUKAN KECURANGAN

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.

Jonathan Alveraldo Bangun

Zahran Alvan Putra Winarko

Dika Pramudya Nugraha

DAFTAR PUSTAKA

- [1] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to Algorithms (3rd ed.). MIT Press.
- [2] Sedgewick, R., & Wayne, K. (2011). Algorithms (4th ed.). Addison-Wesley Professional.
- [3] Meyer, E. A., & Weyl, E. (2017). CSS: The Definitive Guide: Visual Presentation for the Web (4th ed.). O'Reilly Media.
- [4] Knuth, D. E. (1997). The Art of Computer Programming, Volume 1: Fundamental Algorithms (3rd ed.). Addison-Wesley.
- [5] Microsoft Azure. (n.d.). *Azure for Students*. Diakses dari <https://azure.microsoft.com/en-us/free/students/>
- [6] Competitive Programming Algorithms. (n.d.). *Lowest Common Ancestor - Binary Lifting*. Diakses dari https://cp-algorithms.com/graph/lca_binary_lifting.html
- [7] TypeScript. (n.d.). *The starting point for learning TypeScript*. Diakses dari <https://www.typescriptlang.org/docs/>